Scale-Up Analytics: DuckDB vs Apache Spark

Comparing Scale-Up and Scale-Out Approaches
on the TPC-H 100 GB Benchmark

Franciszek Job Przemysław Popowski

June 8, 2026

Abstract

This report presents an empirical comparison of two approaches to large-scale data analytics: **scale-up** using DuckDB on a single HPC node of the Ares cluster (PLGrid) and **scale-out** using Apache Spark on an AWS EMR cluster. Both environments are allocated an equivalent amount of RAM (≈128 GB). The benchmark is based on the TPC-H dataset at scale factor 100 (approximately 100 GB of input data in Parquet format). Six representative SQL queries (Q1, Q3, Q5, Q6, Q10, Q21) were executed, each in 4 runs (the first *cold* run was discarded; the remaining 3 warm runs were used in the analysis). DuckDB achieved execution times in the range of 12–31 seconds, while Spark ranged from 3 to 56 seconds. Each engine won in exactly 3 out of 6 queries: Spark dominated in scans and simple aggregations (Q1, Q6, Q10), while DuckDB outperformed in complex joins and correlated subqueries (Q3, Q5, Q21).

Contents

1 Introduction

As the volume of data generated by organizations continues to grow, choosing the right analytical processing architecture becomes a critical engineering decision. The traditional *scale-out* approach, represented by systems such as Apache Hadoop/Spark, distributes the workload across multiple compute nodes. An alternative is the *scale-up* approach — using a single, powerful server with a large amount of RAM and a modern analytical engine.

1.1 Motivation

The increasing availability of "fat node" servers (256–1024 GB RAM) and the emergence of high-performance in-memory engines such as DuckDB [1] have made the scale-up approach increasingly attractive. DuckDB implements vectorized columnar query execution and is capable of efficiently processing hundreds of gigabytes of data without the need to manage a cluster.

1.2 Research Question

This report addresses the question: *can a single HPC node running DuckDB compete with an Apache Spark cluster, given equivalent memory resources?* The hypothesis is that for data fitting in RAM, DuckDB will be comparable to or faster than Spark, due to the absence of network communication overhead and cluster management costs.

1.3 Scope

- Generation and processing of TPC-H data at scale factor 100 (≈100 GB)
- Environment setup: DuckDB on a PLGrid HPC node and Spark on AWS EMR
- Execution of 6 benchmark queries (Q1, Q3, Q5, Q6, Q10, Q21)
- Analysis of results, costs, and development of a decision framework

2 Test Environment

2.1 HPC Node — Scale-Up (DuckDB)

The scale-up experiments were conducted on a compute node of the Ares cluster at Cyfronet AGH, accessible through the PLGrid infrastructure.

Table 1: HPC Node Specification (Scale-Up)

Parameter	Value
System	Ares, Cyfronet AGH (PLGrid)
SLURM partition	`plgrid`
vCPU count	8
Total RAM	187 GB
RAM for DuckDB	128 GB
Filesystem	NFS Lustre (`/net/pr2`)
Operating system	Linux
Python	3.11
DuckDB	0.10+
Data format	Parquet

2.2 AWS EMR Cluster — Scale-Out (Spark)

The scale-out environment was built on an Amazon EMR cluster, with hardware configuration matched to the HPC node (equivalent RAM).

The entire setup was run within AWS Academy Learner Lab (university account): access restricted to the `us-east-1` region, temporary STS credentials refreshed each session (≈4 h), and academic credit pool instead of card-based billing.

Table 2: AWS EMR Cluster Specification (Scale-Out)

Parameter	Value
EMR version	7.1.0
Region	us-east-1
Master node	m5.xlarge (16 GB RAM, 4 vCPU)
Core nodes (compute)	3× m5.4xlarge (192 GB RAM, 48 vCPU)
Total RAM (core nodes)	192 GB
RAM for Apache Spark	128 GB
Storage	Amazon S3
Apache Spark	3.5
`spark.executor.memory`	16g
`spark.executor.cores`	4
`spark.executor.instances`	8
`spark.sql.shuffle.partitions`	200
Adaptive Query Execution	enabled

2.3 Hardware Equivalence Principle

The key assumption of this benchmark is matching memory resources: both environments have ≈128 GB RAM available for the analytical engine. The EMR master node does not participate in computations and is excluded from the comparison.

3 Test Data — TPC-H 100 GB

3.1 TPC-H Standard

TPC-H [2] is a widely used benchmark for Decision Support Systems. The dataset models an order database of a wholesale supplier with 8 tables of varying sizes. The scale factor (SF) parameter determines the data scale: SF = 1 corresponds to ≈1 GB, and SF = 100 corresponds to ≈100 GB of raw data.

3.2 Data Generation

On the scale-out side (Spark/EMR), data was generated using `tpch-dbgen` (repository `electrum/tpch-dbgen`) compiled directly on the EMR master node. Large tables were split into parts and streamed to S3 (`lineitem` in 3 parts, `-C 3`; `partsupp` in 2 parts, `-C 2`) to fit within the master's local disk space. Raw `.tbl` files (pipe-separated values) were uploaded to an S3 bucket, where Apache Spark converted them to Parquet format with ZSTD compression. On the scale-up side (DuckDB/HPC), an equivalent dataset was generated and used independently on the Ares compute node.

Table 3: TPC-H Table Sizes at SF=100

Table	Rows	Size (.tbl)	Description
`lineitem`	≈600M	≈68 GB	Order line items (primary)
`orders`	≈150M	≈15 GB	Orders
`partsupp`	≈80M	≈10 GB	Part-supplier combinations
`customer`	≈15M	≈2.3 GB	Customers
`part`	≈20M	≈2.3 GB	Parts
`supplier`	≈1M	≈136 MB	Suppliers
`nation`	25	<1 MB	Nations
`region`	5	<1 MB	Regions
Total		≈100 GB	

4 Benchmark Methodology

4.1 Query Selection

Six queries were selected from the official TPC-H set of 22, representing different computational patterns:

Table 4: Selected TPC-H Queries

No.	Name	Pattern	Rationale
Q1	Pricing Summary	Aggregation on ≈68 GB	Pure aggregation, ideal for DuckDB
Q3	Shipping Priority	3-table JOIN + grouping	Classic OLAP join
Q5	Local Supplier Volume	6-table JOIN	Complex multi-join
Q6	Revenue Change	Filter + aggregation	IO-bound, simple arithmetic
Q10	Returned Item Rep.	4-table JOIN + filter	Grouped analysis
Q21	Suppliers Waiting	EXISTS / NOT EXISTS	Correlated subqueries

5 Results

5.1 Execution Times

Table 5: TPC-H Query Execution Times — DuckDB vs Spark [seconds]

	DuckDB (HPC)				Spark (EMR)				Speedup
Query	min	mean	max	std	min	mean	max	std	Duck/Spark
Q1	17.90	18.49	19.38	0.78	8.89	9.35	9.89	0.50	1.98
Q3	15.13	15.59	15.91	0.41	15.94	16.16	16.40	0.23	0.96
Q5	16.03	17.20	19.09	1.65	26.46	26.79	27.11	0.33	0.64
Q6	11.56	12.04	12.57	0.51	2.91	3.07	3.34	0.23	3.92
Q10	25.01	25.17	25.26	0.14	13.97	14.01	14.08	0.06	1.80
Q21	29.92	31.30	33.30	1.77	54.71	55.36	55.75	0.57	0.57

5.2 Observations

The results show clear variation depending on the nature of the query. DuckDB wins in 3 out of 6 cases (Q3, Q5, Q21), while Spark wins in the remaining 3 (Q1, Q6, Q10).

- **Q6** — largest Spark advantage (4× faster, 3.07 s vs 12.04 s). Simple range-predicate filtering on `lineitem` — Spark efficiently scans columnar data from S3, and Adaptive Query Execution (AQE) dynamically optimizes execution and reduces the number of partitions.
- **Q1** — Spark 2× faster (9.35 s vs 18.49 s). Full scan of `lineitem` with aggregation — Spark leverages the combined parallelism of the cluster (core nodes total 48 vCPU), whereas DuckDB is limited to the 8 vCPU of a single node.
- **Q10** — Spark clearly faster (14.01 s vs 25.17 s, ratio 1.80×). 4-table JOIN with filtering — Spark effectively distributes join work across nodes in the distributed environment.
- **Q3** — tie (16.16 s vs 15.59 s, difference <4% in DuckDB's favor). Classic OLAP 3-table JOIN — despite a large disparity in available hardware resources, a standalone DuckDB node handles this query as fast as the entire Spark cluster.

- **Q5** — DuckDB over 50% faster (17.20 s vs 26.79 s). 6-table JOIN with small dimension tables — DuckDB efficiently buffers small tables in local RAM, avoiding the significant network communication overhead incurred by Spark.
- **Q21** — largest DuckDB advantage (31.30 s vs 55.36 s, Spark overhead 1.77×). `EXISTS`/`NOT EXISTS` subqueries — DuckDB's optimizer converts these into efficient local *semi-join* and *anti-join* operations. In Spark, these operations are particularly costly due to the requirement for distributed join processing.

5.3 Charts

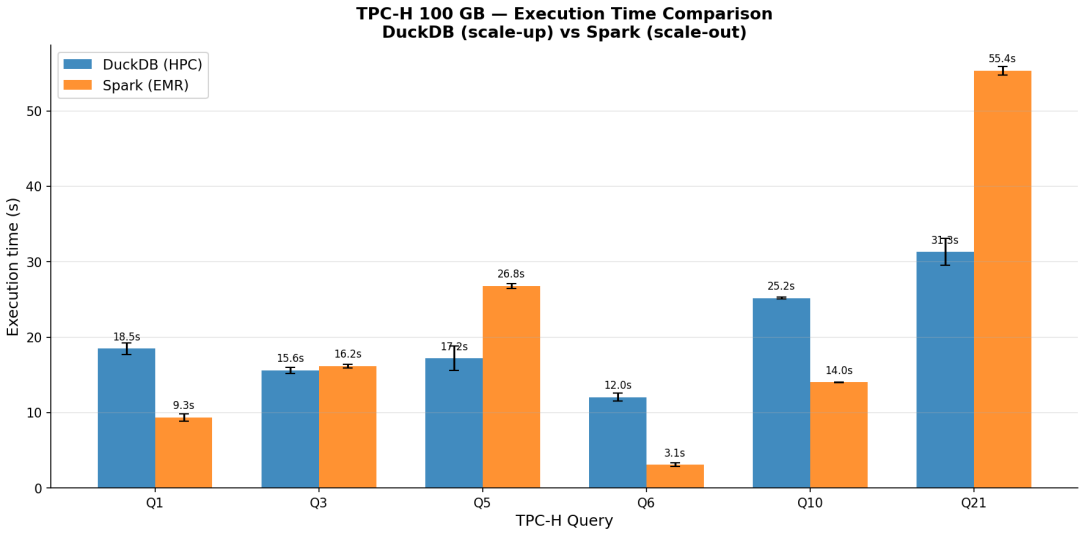

Figure 1: Execution time comparison — DuckDB vs Spark

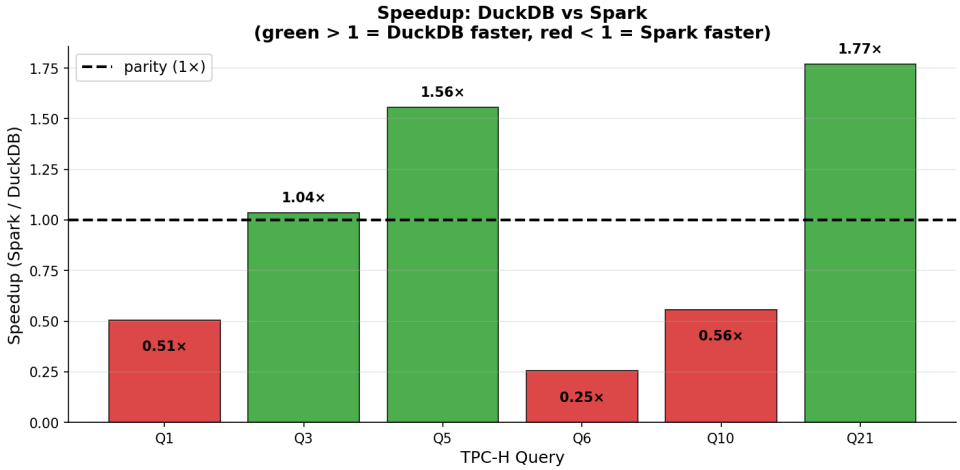

Figure 2: Speedup ratio DuckDB vs Spark per query (Duck/Spark >1 means DuckDB faster)

6 Analysis

6.1 DuckDB Advantages (Scale-Up)

DuckDB executes analytical queries using a *push-based vectorized execution* model with columnar data layout in memory. Key advantages in the context of this benchmark:

1. **No network overhead** — data resides in local node memory, no costly network shuffle operations
2. **SIMD vectorized operations** — processing batches of rows rather than individual tuples
3. **Lazy materialization** — DuckDB avoids copying columns not used in later stages of the query plan
4. **Low management overhead** — no resource scheduler, master node, ZooKeeper, etc.

6.2 Spark Advantages (Scale-Out)

Apache Spark provides value in different scenarios:

1. **Horizontal scalability** — ability to add nodes when data exceeds the RAM of a single server
2. **Fault tolerance** — automatic task re-execution after node failure
3. **Distributed joins** — for very complex plans with multiple large tables, Spark can better distribute the workload
4. **Ecosystem** — Spark Streaming, MLlib, GraphX

7 Cost Analysis

Table 6: Infrastructure Cost Comparison

Option	Estimated cost/h	RAM	Operational overhead
HPC PLGrid (grant)	≈ \$0.00	187 GB	low
AWS EMR (m5.xlarge + 3× m5.4xlarge)	≈ \$2.50	192 GB	high

For datasets that fit entirely in the RAM of a single node (utilization below 80% of available memory), the *scale-up* approach (DuckDB) proves not only more performant but also significantly cheaper and simpler to manage. The EMR cluster (*scale-out*) introduces additional, often overlooked costs and overheads:

- environment startup time (≈20 minutes)
- cost of maintaining the *master* node, which performs no direct compute tasks

- data transfer costs and API call costs to Amazon S3
- time spent managing distributed configuration

Note: Strict lifecycle management of the cluster is essential. Leaving the described AWS EMR environment (m5 instance configuration) in idle (`WAITING`) state over an entire weekend (48 hours) will generate a wasteful cost of approximately **$120**. Automatic cluster termination after benchmark completion is critical from a budget perspective.

8 Conclusions

1. **DuckDB is competitive with Apache Spark** given equivalent memory resources for datasets of ≈100 GB. Query execution times of 12–31 seconds are acceptable for interactive analysis.
2. **Scale-up has a cost and operational advantage** for data fitting in node RAM: no need to manage a cluster, lower setup time, no risk of leaving a running (and costly) cluster idle.
3. **Scale-out (Spark/EMR) is justified** when data exceeds the RAM of a single node, fault tolerance is required, or the workload extends beyond SQL analytics (streaming, ML).
4. **Recommendation**: start with DuckDB on a single node. Migrate to Spark when data exceeds ≈1 TB or requirements arise that DuckDB cannot fulfill.

References

[1] Raasveldt, M., & Mühleisen, H. (2019). *DuckDB: an Embeddable Analytical Database.* SIGMOD '19: International Conference on Management of Data.

[2] Transaction Processing Performance Council (2023). *TPC-H Benchmark Specification, Version 3.0.1.* http://www.tpc.org/tpch/

[3] Zaharia, M., et al. (2016). *Apache Spark: A Unified Engine for Big Data Processing.* Communications of the ACM, 59(11), 56–65.